Министерство образования Республики Беларусь
Учреждение образования
«БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ИНФОРМАТИКИ И РАДИОЭЛЕКТРОНИКИ»

Кафедра информатики

ОТЧЕТ
О ЛАБОРАТОРНОЙ РАБОТЕ № 2
РАБОТА С ФАЙЛАМИ. СИНХРОННЫЙ, АСИНХРОННЫЙ ВВОД-ВЫВОД И ОТОБРАЖЕНИЕ ФАЙЛОВ В ПАМЯТЬ

по дисциплине «Операционные среды и системное программирование»

Выполнил ст. гр. 453504 Л. Д. Масалова

Проверил старший препод. каф. Информатики А. Д. Сыс

Минск 2026

1 ЦЕЛЬ РАБОТЫ

Изучить средства операционной системы Windows для работы с файлами и организации ввода-вывода.

Получить практические навыки:

– открытия, создания и закрытия файлов;

– чтения и записи данных посредством Windows API;

– работы с файловыми дескрипторами HANDLE;

– выполнения синхронных операций ввода-вывода

– выполнения асинхронных операций ввода-вывода;

– использования структуры OVERLAPPED;

– ожидания завершения операций ввода-вывода;

– отображения файлов в виртуальную память процесса;

– обработки больших файлов;

– организации прямого доступа к данным в файле;

– контроля нескольких одновременно выполняемых операций ввода-вывода;

– оценки производительности различных способов работы с файлами.

2 ЗАДАНИЕ

Индивидуальный вариант №15: Обработка большого файла частичными отображениями.

Необходимо обработать файл, размер которого условно считается слишком большим для отображения целиком.

Файл обрабатывать последовательными окнами фиксированного размера.

Исследовать размеры окна:

– 1 МБ;

– 16 МБ;

– 64 МБ.

Для каждого окна вычислять контрольную сумму.

Необходимо учитывать требования Windows к выравниванию смещения отображения.

3 РЕЗУЛЬТАТ РАБОТЫ

3.1 Описание предметной области и задачи

В рамках лабораторной работы исследуются системные механизмы операционной системы Windows для работы с файловой подсистемой и

организацией высокопроизводительного ввода-вывода. Предметной областью является обработка больших массивов бинарных данных, размер которых условно или физически превышает объем доступного непрерывного адресного пространства процесса.

Для решения поставленной задачи используется технология отображения файлов в память (Memory-Mapped Files). Данный механизм позволяет связать содержимое файла на диске с диапазоном виртуальных адресов процесса, исключая необходимость промежуточного копирования данных в буферы пользовательского режима, характерного для традиционных функций ReadFile() и WriteFile(). В соответствии с вариантом №15 исследуется метод частичного отображения (скользящего окна), при котором файл обрабатывается последовательными блоками фиксированного размера (1 МБ, 16 МБ, 64 МБ) с расчетом контрольной суммы для контроля целостности данных.

3.2 Описание алгоритма обработки файла частичными отображениями

Функционирование разработанного приложения начинается с этапа формирования тестовых данных, в ходе которого программа запрашивает у пользователя требуемый объем файла и средствами Windows API через функции CreateFileA() и WriteFile() генерирует на накопителе бинарный файл заданного размера с детерминированным заполнением байтами. Далее выполняется инициализация механизма проецирования: файл открывается для чтения, системной функцией GetFileSizeEx() определяется его точный 64-битный размер, после чего вызовом CreateFileMappingA() в ядре операционной системы создается безымянный объект отображения с атрибутом защиты памяти PAGE_READONLY.

Перед началом обхода данных в обязательном порядке производится проверка системного выравнивания: функция GetSystemInfo() считывает базовую гранулярность распределения памяти операционной системы, составляющую 64 КБ, и выполняется проверка кратности выбранного размера окна этой величине, что исключает возникновение ошибок позиционирования смещения. Основная обработка осуществляется в цикле путем последовательного перемещения скользящего окна от начала файла до его конца. На каждом шаге очередной блок проецируется в адресное пространство процесса с помощью функции MapViewOfFile(), которая возвращает прямой указатель на массив байтов. Приложение подсчитывает контрольную сумму содержимого окна путем сложения числовых значений всех его байтов, сразу же освобождает виртуальные адреса вызовом UnmapViewOfFile() для предотвращения исчерпания памяти и сдвигает указатель смещения на величину обработанного фрагмента.

Завершающий этап включает освобождение занятых системных ресурсов путем закрытия дескрипторов объекта отображения и исходного

файла с помощью вызовов CloseHandle(). Для контроля целостности данных вычисленные итоговые контрольные суммы, полученные при различных конфигурациях окон, сопоставляются между собой, подтверждая корректность и полноту считывания всего объема файла.

Схема алгоритма обработки файла частичными отображениями представлена на рисунке 1.

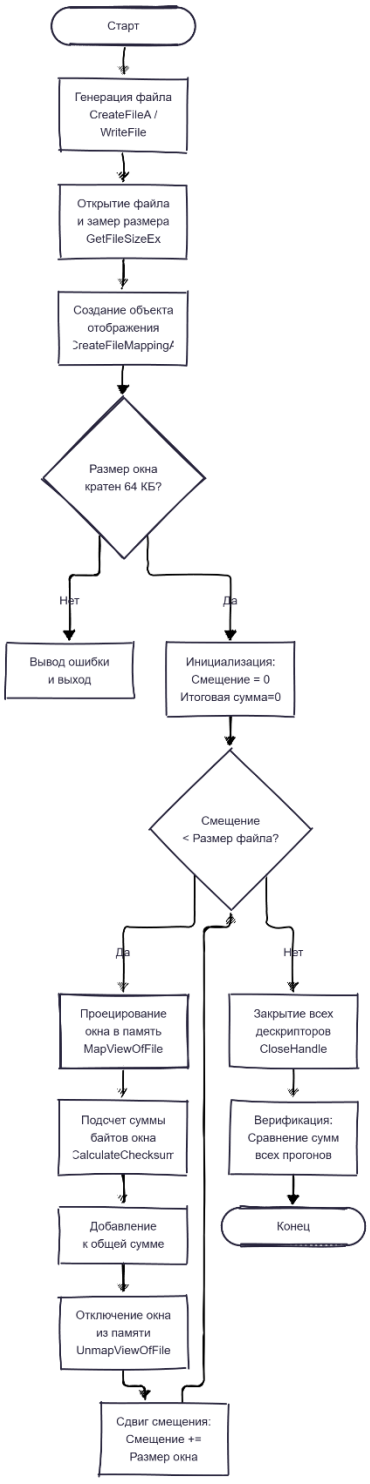


Рисунок 1 – Схема перемещения скользящего окна отображения по файлу

3.3 Перечень использованных функций Windows API

В ходе реализации программы были задействованы следующие системные функции Windows API:

– CreateFileA() – создание нового тестового файла на диске, а также открытие существующего файла для последующего отображения;

– WriteFile() – синхронная блочная запись сгенерированных тестовых данных на накопитель;

– GetFileSizeEx() – получение точного 64-битного размера целевого файла;

– GetSystemInfo() – получение системных характеристик архитектуры Windows, включая величину гранулярности выделения памяти;

– CreateFileMappingA() – создание системного объекта ядра «отображение файла» с атрибутом защиты памяти PAGE_READONLY;

– MapViewOfFile() – проецирование заданного фрагмента файла с указанным смещением в виртуальное адресное пространство процесса;

– UnmapViewOfFile() – удаление проекции представления файла из адресного пространства процесса;

– CloseHandle() – корректное закрытие файловых дескрипторов и дескрипторов объектов отображения;

– GetLastError() – запрос числового кода системной ошибки при возникновении сбоя;

– FormatMessageA() – преобразование системного кода ошибки в локализованное текстовое описание;

– QueryPerformanceFrequency() и QueryPerformanceCounter() – аппаратное высокоточное измерение временных интервалов выполнения операций;

– SetConsoleCP() и SetConsoleOutputCP() – переключение кодировки консоли ввода/вывода на UTF-8 для корректной работы с кириллицей.

3.3 Основные фрагменты исходного кода

Программный комплекс разделен на независимые модули, отвечающие за генерацию файлов, замер времени, обработку ошибок и алгоритм отображения данных.

Ключевой алгоритм обработки файла скользящим окном реализован в функции ProcessFileWithWindows, представленной в листинге 1. В цикле выполняется расчет границ текущего окна, разбиение 64-битного смещения на старшую и младшую 32-битные части, проецирование фрагмента через MapViewOfFile(), суммирование байтов и обязательное освобождение адресов функцией UnmapViewOfFile().

```
     ProcessResult ProcessFileWithWindows(const std::string& filename,
size_t windowSize) {
          ProcessResult result;
          SYSTEM_INFO sysInfo;
          GetSystemInfo(&sysInfo);
          if (windowSize % sysInfo.dwAllocationGranularity != 0) return
result;

          HANDLE hFile = CreateFileA(filename.c_str(), GENERIC_READ,
FILE_SHARE_READ,
                                     NULL, OPEN_EXISTING,
FILE_ATTRIBUTE_NORMAL, NULL);
          if (hFile == INVALID_HANDLE_VALUE) {
PrintError("CreateFile"); return result; }

          LARGE_INTEGER fileSize;
          GetFileSizeEx(hFile, &fileSize);
          uint64_t totalBytes =
static_cast<uint64_t>(fileSize.QuadPart);

          HANDLE hMapping = CreateFileMappingA(hFile, NULL,
PAGE_READONLY, 0, 0, NULL);
          if (!hMapping) { CloseHandle(hFile); return result; }

          HighResolutionTimer timer;
          timer.Start();
          uint64_t currentOffset = 0;
          size_t windowIndex = 0;
          uint32_t combinedChecksum = 0;

          while (currentOffset < totalBytes) {
              uint64_t bytesRemaining = totalBytes - currentOffset;
              size_t currentWindowBytes = (bytesRemaining < windowSize)
                                          ?
static_cast<size_t>(bytesRemaining) : windowSize;

              DWORD offsetHigh = static_cast<DWORD>(currentOffset >>
32);
              DWORD offsetLow  = static_cast<DWORD>(currentOffset &
0xFFFFFFFF);

              char* mappedPtr = static_cast<char*>(MapViewOfFile(
                  hMapping, FILE_MAP_READ, offsetHigh, offsetLow,
currentWindowBytes));
              if (!mappedPtr) { PrintError("MapViewOfFile"); break; }

              combinedChecksum += CalculateChecksum(mappedPtr,
currentWindowBytes);
              UnmapViewOfFile(mappedPtr);
```

```
            currentOffset += currentWindowBytes;
            windowIndex++;
        }
        timer.Stop();

        CloseHandle(hMapping);
        CloseHandle(hFile);

        result.success = true;
        result.elapsedMilliseconds = timer.GetMilliseconds();
        result.totalChecksum = combinedChecksum;
        result.windowsProcessed = windowIndex;
        return result;
    }
```

Листинг 1 – Обработка файла частичными отображениями

Модуль вычисления контрольной суммы окна представлен в листинге 2. Алгоритм выполняет последовательное поэлементное суммирование числовых значений байтов отображенной области памяти.

```
static uint32_t CalculateChecksum(const char* data, size_t size) {
    uint32_t sum = 0;
    for (size_t i = 0; i < size; ++i) {
        sum += static_cast<uint8_t>(data[i]);
    }
    return sum;
}
```

Листинг 2 – Вычисление контрольной суммы отображенного фрагмента

3.4 Результаты экспериментального исследования

Экспериментальное исследование проводилось на сгенерированном тестовом бинарном файле объемом 100 МБ (104 857 600 байт). Исследовалась скорость последовательной обработки файла при использовании различных размеров скользящего окна отображения: 1 МБ, 16 МБ и 64 МБ. Для каждого размера окна было выполнено по три независимых прогона, по результатам которых рассчитано среднее время выполнения и достигнутая пропускная способность ввода-вывода (в МБ/с).

Сводные результаты экспериментальных измерений представлены в таблице 1.

Таблица 1 – Результаты исследования производительности при различных размерах окна отображения

Размер окна	Прогон 1, мс	Прогон 2, мс	Прогон 3, мс	Среднее время, мс	Число окон	Скорость, МБ/с
1 МБ	239,02	239,01	246,42	241,49	100	414,10
16 МБ	241,05	234,16	235,73	236,98	7	421,98
64 МБ	240,63	233,38	238,44	237,48	2	421,08

Пример вывода работы программы в консоли с потоками приведен на рисунке 2.

```
>>> Исследование для размера окна: 1 МБ <<<
  [Прогон 1/3]... Время: 239.02 мс | Окон: 100 | Хэш: 0x1ce00000
  [Прогон 2/3]... Время: 239.01 мс | Окон: 100 | Хэш: 0x1ce00000
  [Прогон 3/3]... Время: 246.42 мс | Окон: 100 | Хэш: 0x1ce00000
--------------------------------------------------
ИТОГ для окна 1 МБ | Среднее время: 241.49 мс | Скорость: 414.10 МБ/с
--------------------------------------------------

>>> Исследование для размера окна: 16 МБ <<<
  [Прогон 1/3]... Время: 241.05 мс | Окон: 7 | Хэш: 0x1ce00000
  [Прогон 2/3]... Время: 234.16 мс | Окон: 7 | Хэш: 0x1ce00000
  [Прогон 3/3]... Время: 235.73 мс | Окон: 7 | Хэш: 0x1ce00000
--------------------------------------------------
ИТОГ для окна 16 МБ | Среднее время: 236.98 мс | Скорость: 421.98 МБ/с
--------------------------------------------------

>>> Исследование для размера окна: 64 МБ <<<
  [Прогон 1/3]... Время: 240.63 мс | Окон: 2 | Хэш: 0x1ce00000
  [Прогон 2/3]... Время: 233.38 мс | Окон: 2 | Хэш: 0x1ce00000
  [Прогон 3/3]... Время: 238.44 мс | Окон: 2 | Хэш: 0x1ce00000
--------------------------------------------------
ИТОГ для окна 64 МБ | Среднее время: 237.48 мс | Скорость: 421.08 МБ/с
```

Рисунок 2 – Фрагмент вывода результатов работы программы в консоли

Анализ экспериментальных данных показывает, что во всех сериях испытаний итоговая контрольная сумма файла строго совпала и составила 0x1ce00000, что подтверждает полную корректность алгоритма и отсутствие потерь данных при сдвиге представлений.

При увеличении размера окна отображения с 1 МБ до 16 МБ наблюдается сокращение среднего времени обработки файла с 241,49 мс до 236,98 мс, что сопровождается ростом скорости чтения с 414,10 МБ/с до 421,98

МБ/с. Данное ускорение обусловлено уменьшением количества системных вызовов к ядру операционной системы: вместо 100 циклов проецирования и отключения представлений программа выполнила всего 7 операций.

При дальнейшем переходе к окну размером 64 МБ среднее время составило 237,48 мс при скорости 421,08 МБ/с, что практически идентично показателям для окна в 16 МБ. Это свидетельствует о том, что при достижении определенного объема окна накладные расходы на вызовы MapViewOfFile() и UnmapViewOfFile() становятся пренебрежимо малы, а ключевым фактором, ограничивающим производительность, становится физическая пропускная способность подсистемы памяти и дискового накопителя при считывании страниц файла.

ВЫВОД

В ходе выполнения лабораторной работы были изучены низкоуровневые механизмы операционной системы Windows для организации высокопроизводительного ввода-вывода и работы с файловой системой средствами системных вызовов Windows API. На практике было разработано консольное приложение, реализующее технологию частичного отображения больших файлов в виртуальное адресное пространство процесса (Memory-Mapped Files) методом скользящего окна с проверкой системного выравнивания и вычислением контрольной суммы.

Экспериментальное исследование подтвердило высокую надежность и корректность предложенного подхода: во всех сериях испытаний итоговая контрольная сумма файла полностью совпала, доказав отсутствие потерь данных на стыках последовательно проецируемых окон. Сравнительный анализ показал, что переход от малого размера окна в один мегабайт к более крупному окну в шестнадцать мегабайт позволил сократить общее время обработки данных на несколько процентов за счет многократного уменьшения числа обращений к ядру операционной системы для проецирования и отключения представлений страниц памяти.

В то же время при дальнейшем увеличении размера окна до шестидесяти четырех мегабайт время выполнения практически перестало изменяться и вышло на плато. Это свидетельствует о том, что при достаточно большом окне накладные расходы на системные вызовы становятся пренебрежимо малыми, а дальнейшая скорость обработки начинает упираться в аппаратную пропускную способность подсистемы оперативной памяти и дискового накопителя. Таким образом, поставленные цели были полностью достигнуты, а метод частичного отображения продемонстрировал свою эффективность для работы с файлами практически неограниченного объема в условиях лимитированной виртуальной памяти процесса.